

in-place. In that case, the in-place dunder methods should create and return a new object, just like the numeric and reflected numeric dunder methods.

We didn't make the `galleons`, `sickles`, and `knuts` attributes read-only, which means they can change. So `WizCoin` objects are mutable. Most of the classes you write will create mutable objects, so you should design your in-place dunder methods to modify the object in-place.

If you don't implement an in-place dunder method, Python will instead call the numeric dunder method. For example, if the `WizCoin` class had no `__imul__()` method, the expression `purse *= 10` will call `__mul__()` instead and assign its return value to `purse`. Because `WizCoin` objects are mutable, this is unexpected behavior that could lead to subtle bugs.

Comparison Dunder Methods

Python's `sort()` method and `sorted()` function contain an efficient sorting algorithm that you can access with a simple call. But if you want to compare and sort objects of the classes you make, you'll need to tell Python how to compare two of these objects by implementing the comparison dunder methods. Python calls the comparison dunder methods behind the scenes whenever your objects are used in an expression with the `<`, `>`, `<=`, `>=`, `==`, and `!=` comparison operators.

Before we explore the comparison dunder methods, let's examine six functions in the `operator` module that perform the same operations as the six comparison operators. Our comparison dunder methods will be calling these functions. Enter the following into the interactive shell.

```
>>> import operator
>>> operator.eq(42, 42)          # "Equal", same as 42 == 42
True
>>> operator.ne('cat', 'dog')    # "Not Equal", same as 'cat' != 'dog'
True
>>> operator.gt(10, 20)          # "Greater Than ", same as 10 > 20
False
>>> operator.ge(10, 10)          # "Greater than or Equal", same as 10 >= 10
True
>>> operator.lt(10, 20)          # "Less Than", same as 10 < 20
True
```